

A Distance Measure Between Attributed Relational Graphs for Pattern Recognition

ALBERTO SANFELIU AND KING-SUN FU, FELLOW, IEEE

Abstract—A method to determine a distance measure between two nonhierarchical attributed relational graphs is presented. In order to apply this distance measure, the graphs are characterized by descriptive graph grammars (DGG). The proposed distance measure is based on the computation of the minimum number of modifications required to transform an input graph into the reference one. Specifically, the distance measure is defined as the cost of recognition of nodes plus the number of transformations which include node insertion, node deletion, branch insertion, branch deletion, node label substitution and branch label substitution. The major difference between the proposed distance measure and the other ones is the consideration of the cost of recognition of nodes in the distance computation. In order to do this, the principal features of the nodes are described by one or several cost functions which are used to compute the similarity between the input nodes and the reference ones. Finally, an application of this distance measure to the recognition of lower case handwritten English characters is presented.

I. INTRODUCTION

NONHIERARCHICAL attributed relational graphs are considered good tools to describe pictures or scenes for pattern recognition [1]. Relational graphs have been used in several applications such as chemical structure description, picture encoding, relational data-base systems for pictures, network representation, switching theory, etc.

In order to recognize a pattern, a scene, or parts of a scene represented by graph, different techniques have been proposed. For example, if we are interested in a perfect matching of two graphs, then graph isomorphism technique [2]–[4] or graph grammars [5], [6] can be used. If we only want a perfect matching of one part of the graph with a complete graph, then a subgraph isomorphism technique has to be used.

In the case of local or structural deformations [7] existing in the input graph, the aforementioned methods cannot be applied, and other techniques have to be used. For local deformations, Tsai and Fu [8] studied the problem by using a combined syntactic-statistical method. For local and structural deformations, several distance measures between graphs have been proposed [10]–[19]. The different methods for computing a distance measure between graphs can

be grouped into the following two categories:

- 1) extract the principal features of each graph, represent them as an n -dimensional vector, and apply Euclidean distance;
- 2) compute the minimum number of modifications required to transform an input graph into the reference one. This can be done in two ways:
 - a) by computing the transformations from an external reference (for example, a grammar),
 - b) by computing the transformations using only the input and the reference graph.

Three general methods, 1), 2a), and 2b), of computing a distance measure are briefly reviewed. Method 1) can only be applied in the case where the graphs are well defined by specific features, for example, number of nodes, number of nodes with one, two, or three connections, number of triangles, etc. These distances measures generally fulfill the properties of a metric. In 2a), the distance measure is computed from an external reference, for example, a grammar. This distance is based on the minimum number of rules that have to be applied to the axiom (or the start graph of the grammar) to reproduce the input graph. This method is typically an error-correcting procedure between a graph and a grammar. An example of this method for the special case of trees can be found in [12]. Another example is the Gernet distance [10], although this distance presents a normalization problem.

Method 2b) is commonly called a matching procedure between two graphs. In this case, we compute the minimum number of direct transformations that have to be applied to modify an input graph into the reference one. The common direct transformations are node insertion, node deletion, branch insertion, branch deletion, node label substitution, branch label substitution, node label merging, and node label splitting. In the case of a tree, this type of distance has polynomial time complexity [11], [17]. For other cases, time complexity has not been reported [10], [12]–[19]. In general, the time complexity is exponential with respect to the number of nodes.

The enumerated distance measures present two important problems when applied to pattern recognition. First, these distance measures only take into account the general structure of the graphs, but they cannot include special features or structures. The absence of this point usually restrains the possible practical applications and

Manuscript received April 19, 1982; revised December 8, 1982. This work was supported in part by the National Science Foundation under Grant ECS 81-19886.

A. Sanfeliu was with the School of Electrical Engineering, Purdue University, West Lafayette, IN 47907. He is now with Instituto de Cibernética, Barcelona, Spain.

K. S. Fu is with the School of Electrical Engineering, Purdue University, West Lafayette, IN 47907.

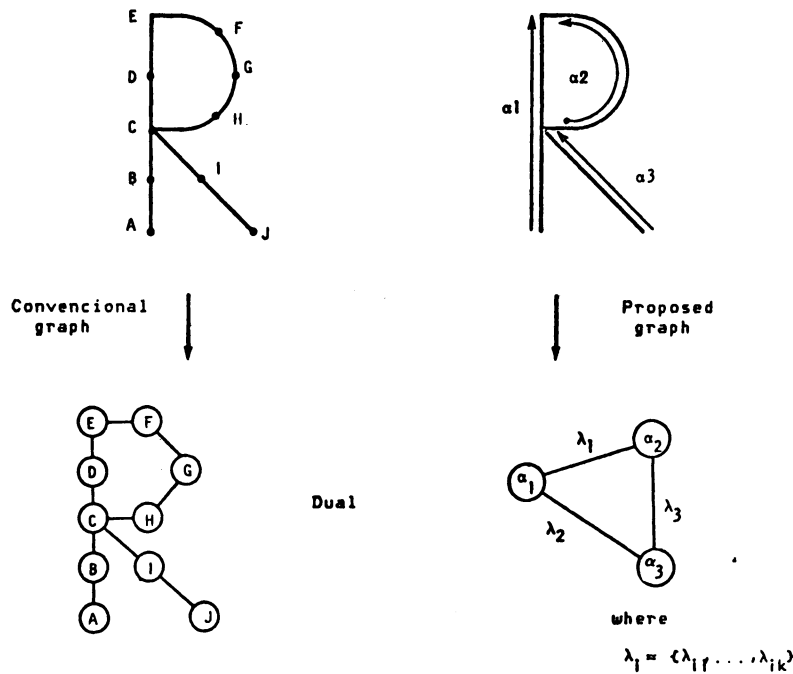


Fig. 1.

sometimes leads to meaningless values of the distance measure. Second, from the theoretical point of view, the node splitting and merging, or at least partial node splitting and merging, have to be considered when using error transformations in the distance measure. If these two errors are not included, then the value of the distance measure cannot be guaranteed to have the minimum in general.

The only distance measure which includes the last two error transformations is Tanaka's measure [18], but because of its definition, its time complexity is very high, since it depends not only on the number of nodes but also on the number of possible function matchings (a summary of this distance measure can be found in [27]).

The proposed distance measure solves these two problems by using the cost of node recognition and the cost of error transformations. The cost of node recognition is computed by means of cost functions which are used to describe special features or structures in a graph. So the first problem is handled by using the cost functions to include special features and the attributes of nodes to describe the general structure of the graph. The second problem is solved by means of the definition of the cost functions. Moreover, the computation of this distance measure is fast when compared with the other ones, since, based on our definition, only a small number of nodes needs to be considered, and the use of node attributes reduces the number of possible node configurations.

This paper is organized in the following way. First, a descriptive graph grammar is presented. Then, the proposed distance measure between nonhierarchical attributed relational graphs is defined, and its properties are explained. Finally, an example of applying this distance measure to the recognition of script handwritten English characters is described.

II. THE PROPOSED DISTANCE MEASURE BETWEEN NONHIERARCHICAL ATTRIBUTED RELATIONAL GRAPHS

In order to apply the proposed distance measure, the graphs could be characterized by a descriptive graph grammars (DGG). The DGG is only used to characterize the graphs but not to compute the distance measure. The DGG is actually used in the pattern segmentation process for finding the nodes and the structure of the graph.

A. Descriptive Graph Grammar

A DGG generates a set of graphs which has the same number of nodes but with different structures. The nodes of these graphs represent the important parts of the patterns. The attributes describe the relations between nodes and are used to generate the structure of these graphs in the grammar derivation. To illustrate that a pattern or scene can be represented by these graphs, an example is given in Fig. 1.

As shown in Fig. 1, the nodes are the essential parts of the character *R* and the attributes characterize the relations between nodes. In this way, a pattern or scene is described by a graph with the minimum number of nodes.

In general, we define a DGG as follows:

$$\text{DGG} = (\Sigma_n, \Sigma_t, \Sigma_\alpha, \Delta, P, S)$$

where

- Σ_n nonterminal alphabet $\{\beta_1 \dots \beta_n\}$,
- Σ_t terminal alphabet $\{a, b, \dots\}$,
- Σ_α node alphabet $\{\alpha_1 \dots \alpha_m\}$.

α_i can be 1) a simple terminal, e.g., *a, b*; 2) a string

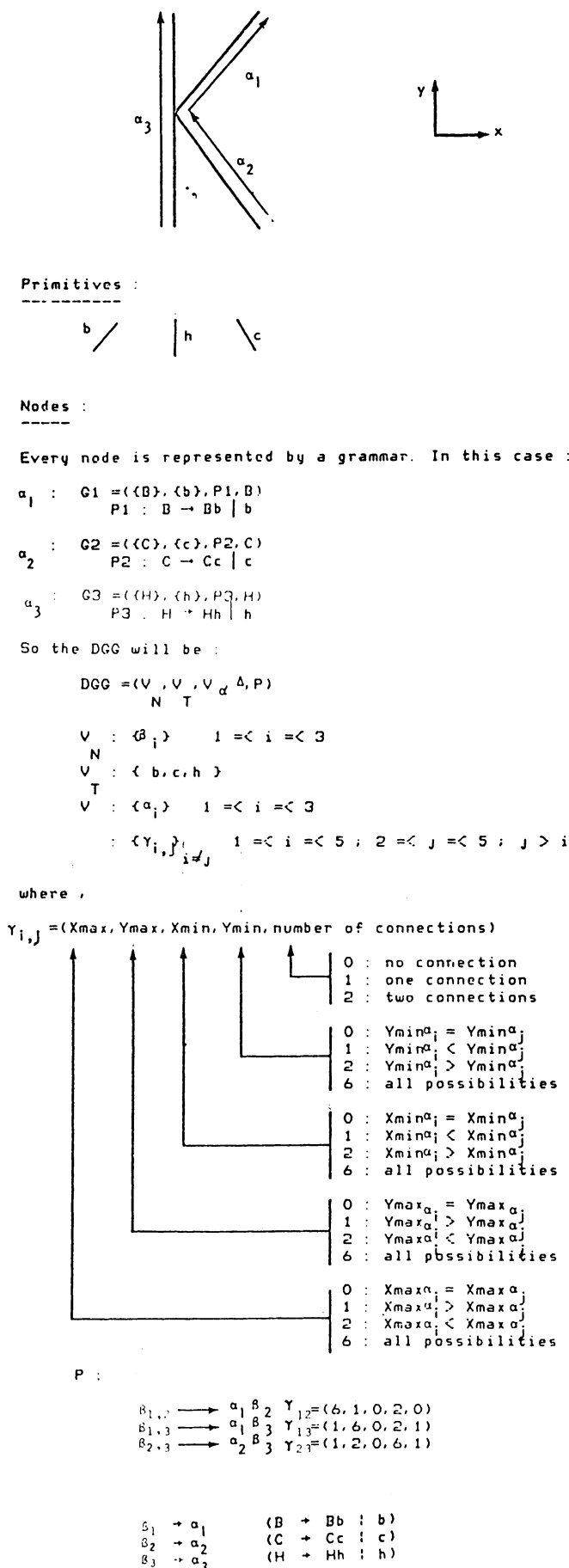


Fig. 2. Character K described by DGG.

generated by a grammar G , e.g., $aaaa$ generated by

$$G = (\{A\}, \{a\}, P, A)$$

$$P: A \rightarrow Aa$$

$$A \rightarrow a;$$

3) an attributed graph; 4) a function or a set of functions which describes a part of the pattern or scene; or 5) a data base

Δ attributed alphabet $\gamma_{i,j}, \gamma_{ij} = \{\gamma_1, \dots, \gamma_k\}$,

P set of production rules: a) $S \rightarrow \beta_{i,j}$, b) $\beta_{i,j} \rightarrow \alpha_i \cdot \beta_j$,

c) $\beta_i \rightarrow \alpha_i$.

Note that the production rule a) is symmetric, that is, it defines at the same time the cases of i, j and j, i .

The language generated by this DGG is

$$L = \{w | \beta \rightarrow w\langle \alpha_j \rangle \text{ where } \beta_i \in \Sigma_n^*, w \in \Sigma_a^*,$$

$$\Sigma_a^* \subset \Sigma_r^*, \gamma_j \in \Delta\}.$$

This language represents all possible paths in the graph. It is important to emphasize that this grammar is only used to describe a set of graphs. An example of a DGG is shown in Fig. 2.

B. Definition of the Proposed Distance Measure

The distance measure defined in this section includes two important characteristics which are not found in other distance measures. The proposed graph distance measure takes into account both the general graph structure and the special features or structures in its computation. Moreover, it considers partial node splitting and node merging. In order to include these two characteristics, this distance measure is obtained by computing the cost of node recognition and the cost of error transformation. The special features or structures are included in the cost of node recognition by means of cost functions, and the general structure is considered in the cost of error transformation as well as in the cost of node recognition. Finally, the partial node splitting and merging are taken into account in the cost functions and in the selection of node configurations.

When an object, scene, or system cannot be represented by a unique graph but can be represented by several graphs, then the cost functions can be computed as follows:

$$C_{xy} = \sum_{k=1}^{m_2} w_{xyk} \cdot f_{xyk} \quad \sum_{k=1}^{m_2} w_{xyk} = 1,$$

where f_{xyk} are the cost of XY (for example, node insertion) for each one of the graphs or correlation factors or a combination of both, and w_{xyk} are the partial weights. The description of these costs is listed in the following.

C_{nr}

Cost of node recognition. It is used to recognize the nodes and to give a similarity value between the input

nodes and the reference ones. This cost is composed by one or several cost functions (f_{nk}) which are used to describe the important features of the nodes or to describe those features that can be used for a specific purpose, for example, classification. Each function will be normalized for a specific range of values.

$C_{ni}, C_{nd}, C_{bi}, C_{bd}$ Number of node insertion, node deletion, branch insertion, and branch deletion, respectively. These costs have to be divided by appropriate factors in order to fit into a similar range of C_{nr} . It has to be emphasized that, although C_{ns} (node label substitution) and C_{bs} (branch label substitution) can be computed by a node (branch) deletion and a node (branch) insertion, their costs can be completely different, and for this reason they are included.

$w_{nr}, w_{ni}, w_{bi}, w_{bd}, w_{ns}, w_{bs}$ Weights of the corresponding coefficients, and their values are generally found by an optimization procedure or heuristically.

In the computation of the distance measure, the minimum value for the possible configurations of nodes is searched, due to the node and branch definition, several possible node configurations can exist. Basically, this distance measure is a matching procedure whose operations are defined in terms of transformations and node costs. To compute the distance, the following two conditions must be fulfilled.

- 1) To apply this distance measure, at least one node of the input graph has to be recognized as a node of the reference one. Otherwise, the input graph will be rejected.
- 2) After applying all needed operations, the transformed graph has to satisfy the attributes of the reference one.

It is important to emphasize that if both graphs are described by a DGG, then the segmentation process can be guided by the DGG.

The formulation of this distance measure is given as follows:

$$d(g_i, g_j) = \min_{\text{configurations}} \{ w_{nr} \cdot C_{nr} + w_{ni} \cdot C_{ni} + w_{nd} \cdot C_{nd} + w_{bi} \cdot C_{bi} + w_{bd} \cdot C_{bd} + w_{ns} \cdot C_{ns} + w_{bs} \cdot C_{bs} \} \quad (1)$$

where

$$w_{nr} + w_{ni} + w_{nd} + w_{bi} + w_{bd} + w_{ns} + w_{bs} = 1$$



Fig. 3. (a) Input graph. (b) Reference graph.

and

$$C_{nr} = \sum_{k=1}^{m_1} w_{nk} f_{nk}, \quad \text{cost of recognition of nodes}$$

with

$$\sum_{k=1}^{m_1} w_{nk} = 1.$$

The cost functions are defined in a way to express the similarity between the input nodes and the reference ones. Fig. 3 will clarify this idea.

In Fig. 3, we assign the following values to the cost function:

reference node $\square$

$f_n = 1,$	if input node is $\square$
$f_n = 0.75,$	if input node is $\sqsupset$ or $\sqsubset$ or $\sqcap$ or $\sqcup$
$f_n = 0.50,$	if input node is $\sqsupset$ or $\sqsubset$ or $\sqcap$ or $\sqcup$
$f_n = 0.25,$	if input node is $_$ or $ $
$f_n = 0,$	otherwise.

The cost functions will always be defined or subjectively assigned in order to be meaningful for the features of the nodes.

The distance computation (1) is a matching procedure. The idea of the method is to find in the input nodes those nodes which are similar to the reference ones. Once we have them, we apply the attributes to the possible configurations of nodes combinations. The configurations of nodes which do not satisfy the attribute specification will be rejected. Then, for each configuration of nodes, we will compute the number of node insertion, node deletion, branch insertion, branch deletion, node label substitution, and branch label substitution. Finally, these values will be put together in formula (1), and $d(g_i, g_j)$ will be computed. The minimum value of all possible configurations of nodes will be $d(g_i, g_j)$.

In general, to apply this distance measure to a specific problem, the nodes of the graph will be selected so that the problem can be completely defined and one or several cost functions can be determined. If a problem cannot be well defined by using only one type of nodes, the problem should be expressed by several graphs. If the separate graphs are not connected, the distance will be computed for each graph in the same way. In the case of interconnection between the graphs, the corresponding correlation has to appear either in the coefficients or in the weights. Finally, the distance coefficients will be computed as the weighted summation (with the correlation factors) of the values of these independent graphs.

The algorithm to compute the distance measure is given as follows.

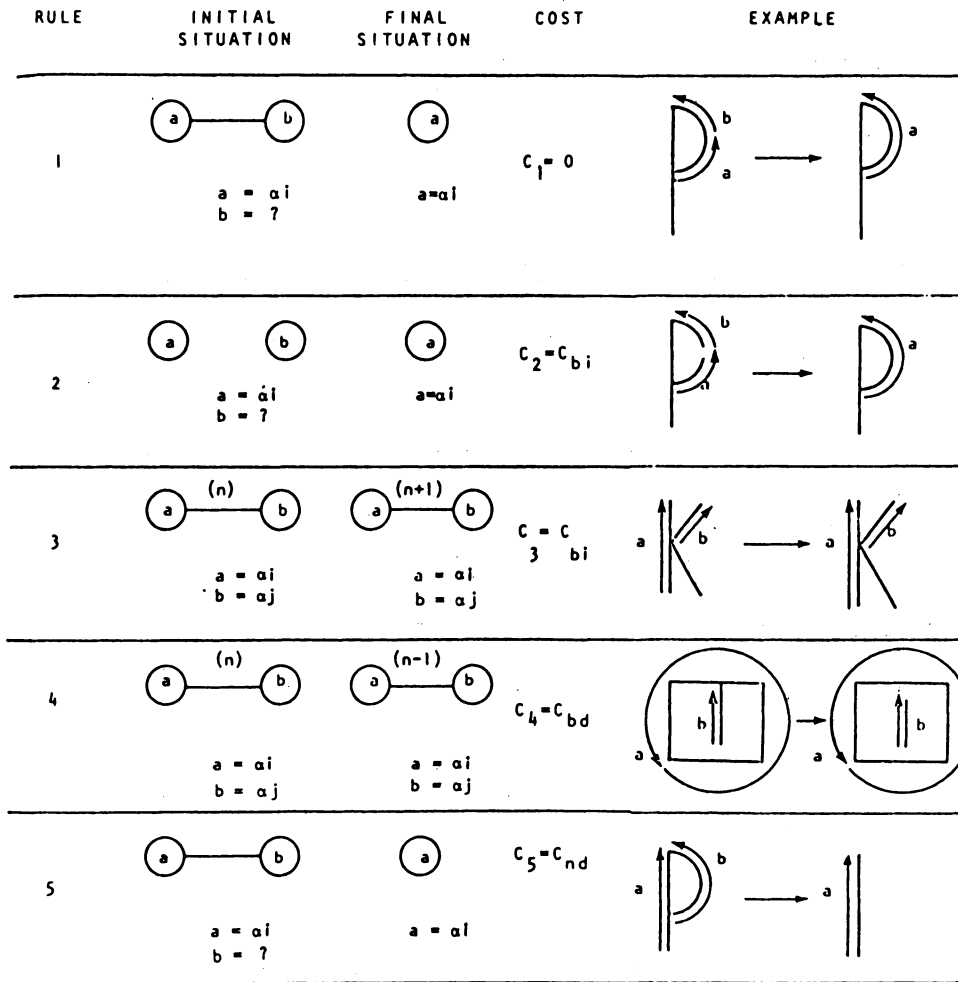


Fig. 4. Operations to transform input graph into reference one.

Algorithm:

- Input** Input graph which is characterized by a DGG and a reference graph which is characterized by a DGG.
- Output** Distance measure between the input and the reference graphs.

Method:

- Step 1** Recognize all possible candidate nodes through their definition (cost function). Compute the associate cost for each node.
- Step 2** Select one possible configuration of nodes. In this step, since one input node can be recognized as several reference nodes, the possible combinations of them have to be studied. For each configuration of nodes the attributes of the nodes are imposed, and the configurations which do not satisfy the attributes are not further considered. If no possible configuration of nodes exists, the graph is rejected.
- Step 3** Make all necessary operations to transform the input graph into the reference one. Calculate the transformation costs by computing each one of the distance coefficients. Compute the $d(g_i, g_j)$

defined in (1). If this cost is greater than a threshold, reject this configuration of nodes.

- Step 4** Check the transformed graph to see if it satisfies the attributes of the reference one. If this is not true, reject this configuration of nodes.
- Step 5** If the computed value is lower than the minimum value of $d(g_i, d_j)$, set the minimum value equal to this value. If no other possible configuration of nodes exists, stop. Otherwise, go to Step 2.

Comments:

1) Step 4 can be omitted if in the application of the operations, the checking is already included, or if the transformed graph does not need any verification.

2) In order to speed up the algorithm, the cost functions and the attributes have to be defined to minimize the possible configurations of nodes.

3) If in the input graph too much structural and local noise exists, the distance that we get may not be meaningful. Therefore, a thresholding of $d(g_i, g_j)$ has to be considered, and the $d(g_i, g_j)$ with value greater than the threshold is rejected. The threshold can be a function of the number of nodes after preprocessing and the number of nodes of

the reference graph, etc. In general,

$$d(g_i, g_j) < f(x).$$

C. Properties

Triangular Inequality: The distance measure $d(g_i, g_j)$, in general, does not satisfy the properties of a metric, because of the definition of the nodes and the branches, and because of the assigned weights. This function always satisfies

$$\begin{aligned} d(g_i, g_i) &= 0 \\ d(g_i, g_j) &> 0, \quad \text{if } g_i \neq g_j. \end{aligned}$$

If the cost functions have the same value from g_i to g_j and from g_j to g_i and the cost of node insertion (branch insertion) is defined in the same way as the cost of node deletion (branch deletion) and

$$w_{ni} = w_{nd}; \quad w_{bi} = w_{bd},$$

then $d(g_i, g_j)$ satisfies

$$d(g_i, g_j) = d(g_j, g_i).$$

Moreover, if at least one configuration of nodes is not rejected, then $d(g_i, g_j)$ also satisfies

$$d(g_i, g_j) \leq d(g_i, g_k) + d(g_k, g_j).$$

2) **Partial Node Merging and Splitting:** The partial node merging and splitting are included in the cost functions, since these functions give an indication of the similarity between an input node and the reference one. Partial node merging happens when a part of a pattern or scene, which is represented as a node in a graph, is partially covered by another part, which is also represented by another node in that graph.

III. APPLICATION OF THE PROPOSED DISTANCE MEASURE TO THE RECOGNITION OF LOWERCASE FREE HANDWRITTEN CHARACTERS

In order to illustrate the application of the proposed distance measure, an example of lowercase character recognition is presented. This example was chosen since lowercase characters have the following properties: 1) the character recognition problem can be easily extended to two- or three-dimensional object recognition, and 2) it is easy to obtain character samples.

Character recognition has been studied by many authors [20]–[23], and a number of character readers have been developed by industrial companies [24]. Generally, lowercase characters have been divided into three areas, upper, central, and lower [20], in order to recognize them. In our case, we do not consider these three areas separately.

Method

The lowercase characters are described by DGG's in order to apply the proposed distance measure. The nodes of the graph represent the strokes [20] of a character. Each node is characterized by a grammar. The branches of the

graph are characterized by the attributes between two nodes, and specifically, they represent the connections or no connections between two nodes. Fig. 2 gives an example of an ideal character.

The operations to transform an input graph into the reference one are shown in Fig. 4. Rules 1 and 2 are actually only one rule, but they are separated because of preprocessing. These two rules are used to construct the nodes since in the preprocessing, only some parts of the nodes are recognized. Rules 3, 4, and 5 are branch insertion, branch deletion, and node deletion. In this case, node insertion is not taken into account, since the configurations of nodes of the input graph of which the number of nodes is not the same as that of the reference graph are not considered. The costs C_{bi} (cost of branch insertion), C_{bd} (cost of branch deletion), and C_{nd} (cost of node deletion) are computed as follows:

$$C_{bi} = \frac{N_{bi}}{N_T}$$

$$C_{bd} = \frac{N_{bd}}{N_T}$$

$$C_{nd} = \frac{N_{nd}}{N_T},$$

where

- N_T number of primitives in the character after preprocessing (remember that a character is represented by a graph in which nodes are described by strings of primitives)
- N_{bi} total number of added primitives to connect two nodes (The connection is done by joining together the closest points of both nodes if special points do not exist. Otherwise, the connections have to be made at those points.)
- N_{bd} total number of deleted primitives to separate one node from another
- N_{nd} total number of deleted primitives to delete a node.

Before we explain the cost function of a node, we will describe a node of a graph. A node is described by a grammar which is composed by one or several segments (a segment is a string produced by production type $A \rightarrow Aa|a$). An example of this grammar is shown in Fig. 5, which represents the following grammar:

$$S \rightarrow DFH$$

$$D \rightarrow Dd|d$$

$$F \rightarrow Ff|f$$

$$H \rightarrow Hh|h.$$

In the preprocessing, each of the segments (d, f, h) is recognized individually. Actually, only the first one (d) is recognized as a possible node (afterwards, the other two are connected by Rules 1 and 2). In order to allow noise in the segments, each one of these is recognized if it is inside a cone of 45 degrees aperture. For example, see Fig. 6.

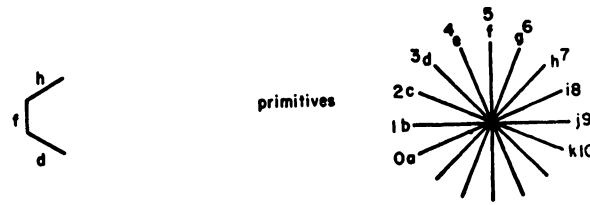


Fig. 5.

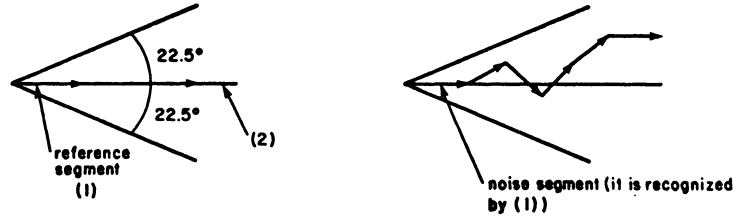


Fig. 6.

In order to express the similarity between a noisy segment and the reference one, the cost function is defined as follows:

- $f_n = 1$, if the noisy segment follows one of the borders of the cone,
 $f_n = 0$, if the noisy segment follows the line (2),
 f_n not defined, if the noisy segment starts outside of the boundary of the cone,
 $f_n > 0$, otherwise.

The cost function is computed for the purpose of expressing the similarity between a noisy segment and the reference one. Its value is

$$C_{nr} = \frac{\sum_i C_{ni}/N}{N_T},$$

where C_{ni} is the cost function for node i ,

$$0 \leq C_{ni} \leq 1,$$

and N is the number of nodes in the reference graph.

The flowchart of the proposed method is shown in Fig. 7. Basically, it follows the same sequence of operations explained in the algorithm in Section II-B. The only difference is that, in this case, no threshold has been imposed. Step 4 of the algorithm in Section II-B is used since the attributes of the nodes do not describe well the relations among the nodes.

Results

The proposed method was applied to the recognition of four lowercase English characters. The characters were b , d , h , and k . These characters were chosen since they are very similar, and it is sometimes difficult even for a human being to distinguish them. In order to classify the characters, the proposed distance measure was computed between the input character and each one of the references. The input character was assigned to the same class as the reference with the minimum distance.

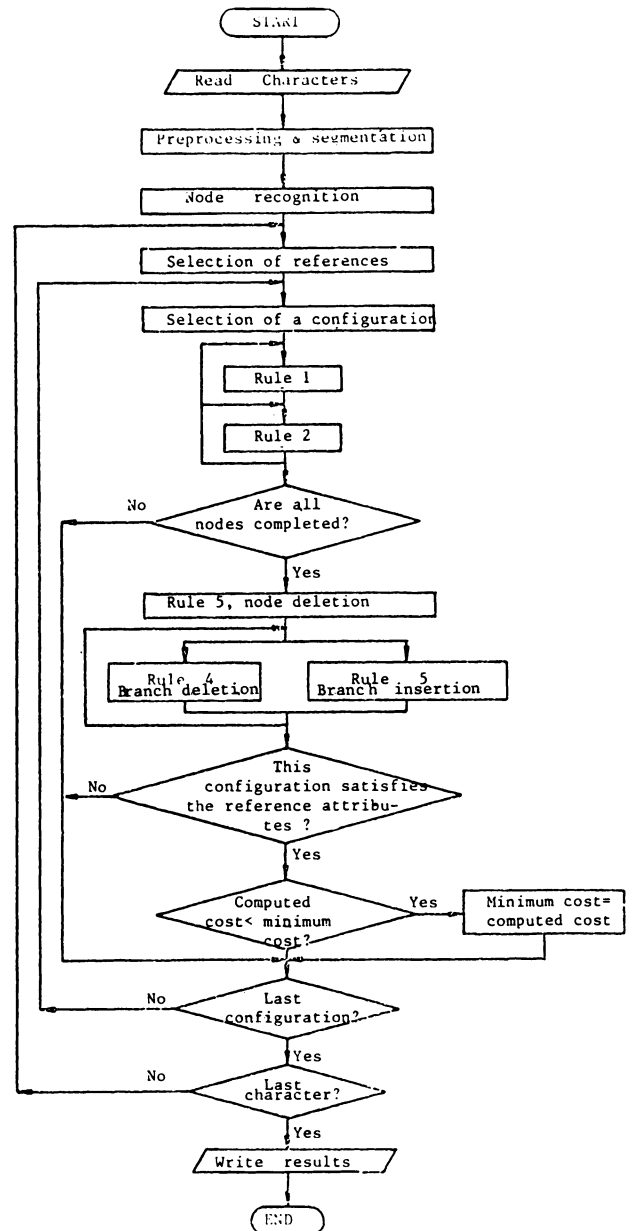
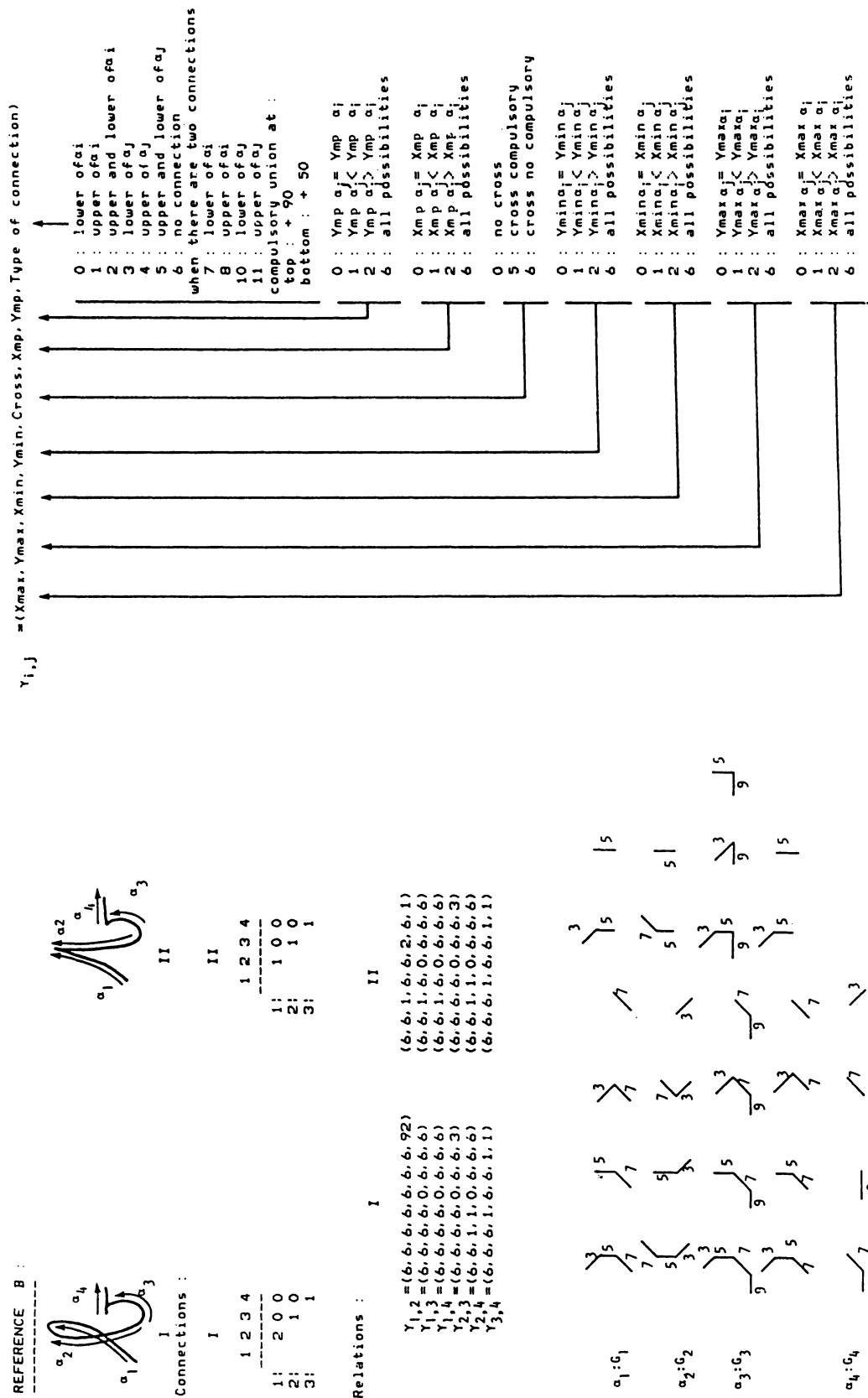


Fig. 7. Flowchart for computation of distance between graphs in recognition of small free handwritten characters.



(b)
 Fig. 8. (a) Reference character b. (b) Description of reference character attributes.

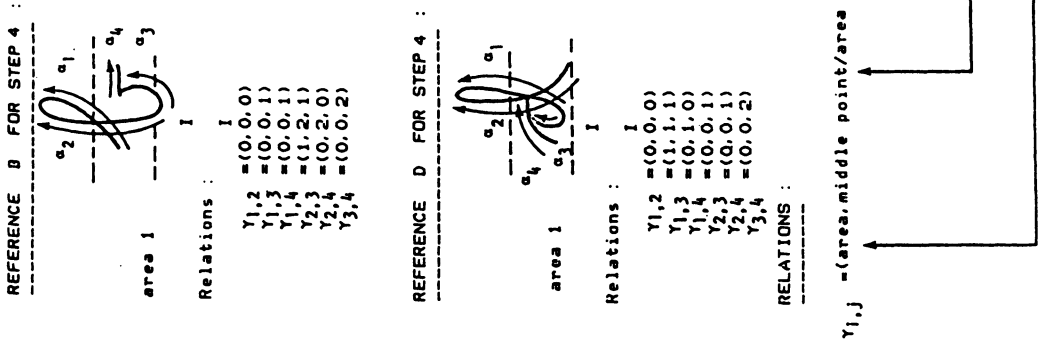


Fig. 9. Used reference for Step 4.

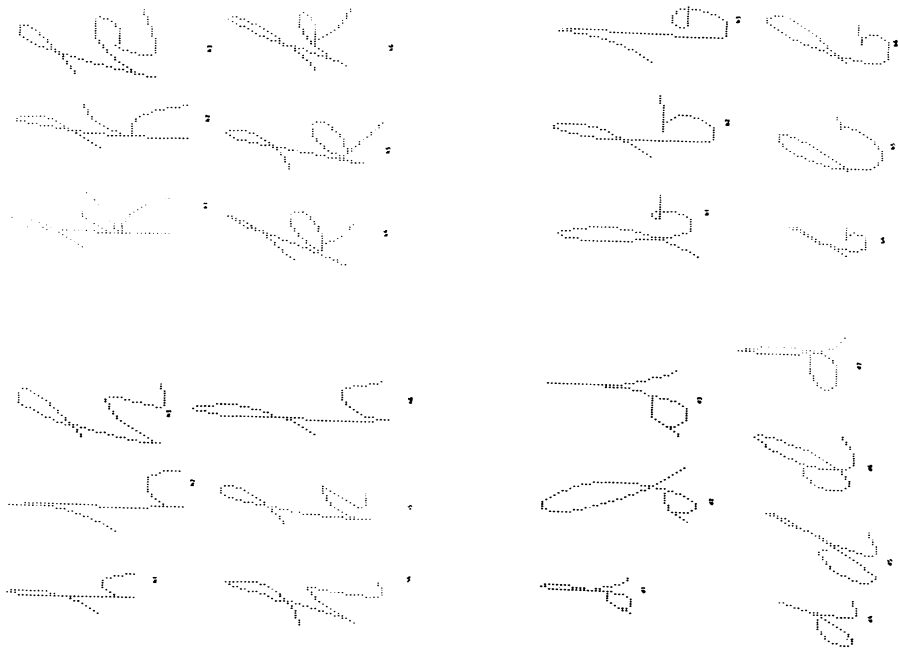


Fig. 10. Input characters.

TABLE I
RESULTS OF 25 SMALL FREE HANDWRITTEN CHARACTERS
(MULTIPLIED BY 100)

	b	d	h	k
b1	* 20.13 *	75.70	23.36	rejected
b2	* 0.01 *	80.07	20.25	48.48
b3	* 0.07 *	66.70	13.43	rejected
b4	* 7.51 *	42.50	17.77	53.87
b5	* 0.01 *	31.22	13.74	rejected
b6	* 0.01 *	79.48	14.09	rejected
d1	rejected	* 4.95 *	rejected	rejected
d2	71.34	* 3.58 *	78.48	rejected
d3	43.95	* 33.30 *	71.12	55.47
d4	81.95	* 28.75 *	106.68	rejected
d5	44.91	* 36.76 *	77.66	rejected
d6	rejected	* 32.83 *	46.17	45.45
d7	55.30	* 22.96 *	39.42	35.97
h1	22.75	rejected	* 9.18 *	rejected
h2	14.88	rejected	* 0.00 *	rejected
h3	42.08	71.51	* 17.25 *	rejected
h4	35.65	57.46	* 10.48 *	rejected
h5	25.98	35.20	* 14.48 *	rejected
h6	19.96	rejected	* 6.59 *	rejected
k1	25.30	38.98	27.72	* 14.70 *
k2	27.45	45.83	14.36	* 9.82 *
k3	38.32	49.26	27.78	* 23.38 *
k4	rejected	48.30	46.22	* 10.14 *
k5	29.73	38.52	24.79	* 9.93 *
k6	rejected	61.72	65.66	* 15.56 *

Fig. 8 shows the references used and the description of their attributes. In Fig. 9 references used in Step 4 of the algorithm in Section II-B are described. For the characters in Fig. 10, the distance measure was computed, and the results are shown in Table I. In this case, the weights of the distance coefficients were $w_{nr} = w_{nd} = w_{bi} = w_{bd} = 0.25$.

Note that, in some of these samples, the number of possible configurations of nodes was high due to the ambiguity of the nodes attributes. This problem could be resolved by using a more complex preprocessing.

IV. CONCLUSION

A distance measure between nonhierarchical attributed relational graphs is proposed in this paper. The distance measure is defined in terms of the cost of node recognition and error transformations. This distance measure includes two characteristics which cannot be found in other distance measures. Specifically, it takes into account not only the general structure of the graph but also the special features or structures, and it also considers partial node splitting and merging. These two considerations are handled in part by means of cost functions which are included in the cost of node recognition. Moreover, these cost functions allow us to describe the principal characteristics of the nodes, and they give a similarity measure between input nodes and the reference ones. Finally, another important feature of this distance measure is the use of relational attributes between nodes which allows us to represent a graph accurately and get a meaningful distance value.

This distance measure has been applied to the recognition of lowercase handwritten characters. In this application, the cost function does not play an important role in the computation of the distance measure, but it is very important as far as the recognition of the nodes is concerned. An application where the cost functions are very

meaningful either for the computation of the distance measure and the recognition of the nodes can be found in [25].

REFERENCES

- [1] K. S. Fu, *Syntactic Methods in Pattern Recognition*. New York: Academic, 1974.
- [2] A. T. Berztsis, "A backtrack procedure for isomorphism of directed graphs," *J. Ass. Comput. Mach.*, vol. 20, July 1973.
- [3] D. G. Corneil and C. C. Gotlieb, "An efficient algorithm for graph isomorphism," *J. Ass. Comput. Mach.*, vol. 17, Jan. 1970.
- [4] D. C. Schmidt and L. E. Druffel, "A fast backtracking algorithm to test directed graphs for isomorphism using distance metrics," *J. Ass. Comput. Mach.*, vol. 23, July 1976.
- [5] H. O. Bunke, "Structural and syntactic methods for the interpretation of electrical schematics," presented at the NSF Workshop on Structural and Syntactic Pattern Recognition, Saratoga Springs, June 22-24, 1981.
- [6] A. Sanfeliu and K. S. Fu, "Tree-graph-grammars for pattern recognitions," in *Proc. 2nd Int. Workshop Graph Grammars*, Osnabruck, Germany, Oct. 4-8, 1982.
- [7] W. H. Tsai and K. S. Fu, "A pattern deformational model and Bayes error-correcting recognition system," in *Proc. Int. Conf. Cybernetics and Society*, Nov. 1978.
- [8] ———, "Subgraph error-correcting isomorphisms for syntactic pattern recognition," *IEEE Trans. Syst., Man, and Cybern.*, vol. 13, Jan./Feb. 1983.
- [9] S. A. Boorman and D. C. Oliver, "Metrics on spaces of finite trees," *J. Math. Psych.*, vol. 10, Oct. 1973.
- [10] D. Gernet, "Measuring the similarity on complex structures by means of graph grammars," *Bull. EATCS*, Feb. 1979.
- [11] S. T. Lu, "A tree-to-tree distance and its application to cluster analysis," *IEEE Trans. Pattern Anal. Machine Intell.*, vol. PAMI-1, pp. 219-224, 1979.
- [12] S. Y. Lu and K. S. Fu, "Error-correcting tree automata for syntactic pattern recognition," *IEEE Trans. Comput.*, vol. C-27, Nov. 1978.
- [13] Y. Kajitani and H. Sakurai, "On distance of graphs defined by use of orthogonality between circuits and cutsets," *Inst. Electron. Comm. Engineers Japan*, Rep. CT-73-30, 1973.
- [14] Y. Kajitani and M. Ueda, "On the metric space of labeled graphs," *Inst. Electron. Comm. Engineers Japan*, Rep. CST-74-115, 1975.
- [15] L. G. Shapiro, "Grouping of relational models," presented at the NSF Workshop on Structural and Syntactic Pattern Recognition, Saratoga Springs, NY, June 22-24, 1981.
- [16] L. G. Shapiro and R. M. Haralick, "Algorithms for inexact matching," in *Proc. 5th Int. Conf. on Pattern Recognition*, vol. 1, Dec. 1980.
- [17] K. C. Tai, "The tree-to-tree correction problem," *Ass. Comput. Mach.*, vol. 26, pp. 422-433, July 1979.
- [18] E. Tanaka, "A metric on graphs and its applications," *IEE Japan*, IP-77-55, Oct. 31, 1977.
- [19] E. Tanaka and Y. Kikuchi, "A metric between pictures," *Dep. Inform. Sci., Utsunomiya Univ., Utsunomiya, Japan*.
- [20] M. Eden and M. Halle, "The characterization of cursive writing," in *Proc. Symp. Information Theory*, C. Cherry, Ed. London: Butterworth, 1961.
- [21] H. Genchi, K. I. Mori, S. Watanabe, and S. Katsuragi, "Recognition of hand written numerical characters for automatic letter sorting," *Proc. IEEE*, vol. 56, pp. 1296-1301, Aug. 1968.
- [22] M. Fujisawa and M. Yasuda, "Directional pattern matching method for hand printed character recognition," presented at the Conf. on Pattern Recognition and Image Processing, Dallas, Tx, Aug. 1981.
- [23] Y. Yamada, T. Kawatani, H. Kaneko and T. Tsutsumeda, "Hand printed numeral recognition by feature concentration method," *Rev. Elec. Comm. Labs*, vol. 26, nos. 11-12, pp. 1537-1546, 1978.
- [24] S. Mori, "Outline of OCR technology in Japan," presented at the NSF Workshop on Structural and Syntactic Pattern Recognition, Saratoga Springs, NY, June 22-24, 1981.
- [25] A. Sanfeliu, K. S. Fu, and J. M. S. Prewitt, "An application of distance between graphs to the analysis of muscle tissue patterns," submitted for publication.
- [26] L. G. Shapiro and R. M. Haralick, "Organization of relational models for scene analysis," *Tech. Rep. CS81011-R*, VPI & SU, Sept. 1981.
- [27] K. S. Fu, "Picture syntax," in *Pictorial Information Systems*, S. K. Chang and K. S. Fu, Eds. New York: Springer-Verlag, 1980.